

Управление «мышлением» агента



Влад Прошинский
AI-консультант, ex-CPO

Обо мне

Влад Прошинский

AI-консультант, ex-CPO

- 10 лет в IT. Запускал IT продукты в Ozon, SOKOLOV, Деловые Линии, RUTUBE, Газпромбанке
- Сейчас помогаю компаниям внедрять AI так по отработанной методологии (Yakov & Partners Playbook) так, чтобы это давало измеримый результат.

Специализация:

- ИИ в маркетинге
- ИИ для отделов продаж
- Автоматизация бизнес-процессов



Канал «ИИ для продакта & CPO»
<https://t.me/AImademyday>

Программа

- Настройки "усилия" (effort) и когда оно нужно
- Бюджеты: лимит шагов/времени/стоимости против runaway/Token DoS.
- Авто-маршрутизация: дешёвый режим → эскалация по триггерам качества

Результат: агент держит качество и не сжигает бюджет

Сколько стоит один запрос вашего агента?

\$0.06

Simple Q&A
один вызов

Один LLM-вызов, прямой ответ

\$2.70–7.20

Multi-step agent
несколько шагов

Chain-of-Thought + Tool calls

\$150+

Runaway agent
один запрос

Агент без остановки, retry loop

Разница в 2500х при одинаковом коде

Обсудим

01

Блок 1 — Настройки «усилия»

Управляем глубиной мышления агента

18 мин

02

Блок 2 — Бюджеты

Защита от runaway и Token DoS

22 мин

03

Блок 3 — Авто-маршрутизация

Дешёвый режим → эскалация по качеству

20 мин

04

Блок 4 — Практика в n8n

Собираем живой workflow

15 мин

БЛОК 1 – НАСТРОЙКИ «УСИЛИЯ»

Thinking: «поверхностно» или «глубоко»

ПОВЕРХНОСТНО

- 1 LLM-вызов — ответ за 1 секунду
- Стоимость: ~\$0.01
- Нет цепочки рассуждений
- Нет вызовов инструментов

ГЛУБОКО

- Chain-of-Thought + Tool calls
- Reflection + Self-correction
- Несколько итераций → 30 сек
- Стоимость: ~\$0.50–2.00

Ключевой вопрос: не «умнее = лучше», а «какое усилие нужно для этой задачи?»

Четыре режима — от рефлекса до рефлексии

~ \$0.003 REFLEXIVE Прямой ответ без рассуждения Пример: «Как вас зовут?»	~ \$0.01 STANDARD Стандартный промпт + 1 вызов Пример: FAQ, классификация	~ \$0.05 DELIBERATE Chain-of-Thought до 5 шагов Пример: Анализ, сравнение вариантов	~ \$0.50+ EXHAUSTIVE Много итераций, reflection, self-correction Пример: Код, сложный reasoning
---	---	---	---

DELIBERATE — агент рассуждает, но ограничен 5 шагами. EXHAUSTIVE — аналог effort: high в OpenAI API

Команды в Claude Code (для разработчиков)

think	think harder	ultrathink
Базовый	Углублённый	Максимальный
Токенов: ~4,000	Токенов: ~10,000	Токенов: ~32,000

Параметр effort в API

Параметр reasoning.effort

"low" – быстро, дёшево, меньше токенов на рассуждение

"medium" – баланс (по умолчанию)

"high" – максимальная глубина reasoning

Аналог в Claude: thinking budget — явный лимит токенов на thinking-блок

```
response = client.responses.create(  
    model="o4-mini",  
    reasoning={  
        "effort": "low"  
        # low / medium / high  
    },  
    input=[{  
        "role": "user",  
        "content": prompt  
    }]  
)
```

Аналог в Claude — max_tokens для thinking блока

```
{  
  "thinking": { "type": "adaptive" },  
  "effort": "max" // low | medium | high | xhigh | max  
}
```

Высокое усилие оправдано только в 20% задач

Нужно

- Многошаговое рассуждение (план проекта, архитектура)
- Код с логическими ошибками
- Юридический / медицинский анализ
- Задача с противоречивыми данными

НЕ нужно

- FAQ-ответы
- Классификация текста
- Форматирование / перевод
- Простое извлечение данных

50–70% запросов не требуют дорогой модели

50–70%

запросов — дешёвая
модель справится

20–30%

реально требуют
frontier-модели

до 80%

снижение стоимости
при умном роутинге

70% — дешёвая модель

30% — Frontier-модель

- Простая классификация и FAQ → ~98% качества на дешёвой модели
- Форматирование, перевод, извлечение данных → аналогично
- Только сложные задачи реально требуют frontier-модели

\$300 в день превратились в \$200 000 в год

Кейс #1 — реальный production

Что пошло не так:

- Retry loops после ошибок инструментов — агент повторял бесконечно
- Параллельные инстансы без лимитов — кратное умножение стоимости
- Tool call overhead на каждом шаге — дополнительные токены
- Контекст re-stacking — история накапливалась без обрезки
- Дорогая модель на простых шагах — отсутствие роутинга

Расчётный бюджет: \$109 500/год → Реальный: \$120 000–200 000/год (разница 5–7х)

Каждый шаг агента дороже предыдущего

Контекстное окно растёт квадратично



API биллится по полному контексту на каждом шаге. При 10 шагах контекст в 8–10х больше первого

max_iterations защищает не только от зависания, но и от роста стоимости

Без memory агент съел в 16 000x больше токенов

Кейс #2 — IBM Materials Science (сгенерировать структуру молекулы и рассуждать о её свойствах)

20 000 000

токенов — наивный агент
(передаёт весь контекст)

VS

1 234

токена — агент
с memory pointers

Разница: 16 234x · Управление контекстом = управление стоимостью

Тупой подход: передаёт полный контекст на каждом шаге → 20 000 000 токенов → падение

Умный подход: хранит ссылки на данные, а не сами данные → успех

Без memory агент съел в 16 000x больше токенов

Кейс #3 — IBM Materials Science (сгенерировать структуру молекулы и рассуждать о её свойствах)

Решение:

Хранить большие данные за пределами контекстного окна, это позволяет модели взаимодействовать с ними только через короткие идентификаторы — указатели (pointers).

Каждый инструмент «зеркалируется» обёрткой, которая состоит из трёх компонентов:

1. модуль, определяющий — это указатель или сырые данные;
2. оригинальный инструмент с неизменённым поведением;
3. постпроцессор, который сохраняет большие выводы в памяти и возвращает инструкции доступа

Без memory агент съел в 16 000x больше токенов

Кейс #3 — IBM Materials Science (сгенерировать структуру молекулы и рассуждать о её свойствах)

1. Можно ли было просто обрезать данные?

Существующие подходы — truncation и summarization — не сохраняют полный вывод, что делает их неприменимыми для workflow, требующих полных данных. В химии нельзя передать следующему инструменту «примерную структуру молекулы» — нужна точная.

2. Применимо ли это только к химии?

Нет — паттерн работает с любым фреймворком, который поддерживает tool context. Для single-agent используется `agent.state`, для multi-agent — `invocation_state`, который разделяется между всеми агентами в Swarm.

Усилием управляют три рычага

Модель

Tier 1 (mini/haiku) vs
Tier 3 (GPT-4o/Opus)

Разница в цене: 17–19x

Выбирается в одном
дропдауне p8n

Глубина reasoning

effort: low /
medium / high

max_tokens для
thinking-блока Claude

Директивный контроль
через API

Количество шагов

max_iterations
в агентной петле

Early stopping при
достижении цели

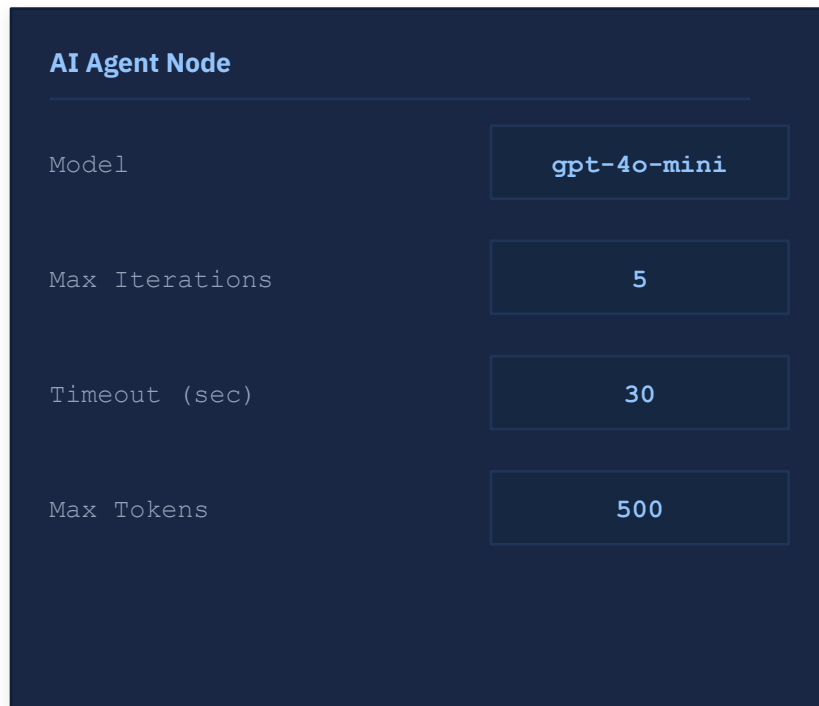
Защита от
quadratic cost growth

Комбинация трёх рычагов даёт полный контроль над соотношением цена / качество

В n8n это настраивается в одном узле

Управление усилием без написания кода

- AI Agent Node → Options:
 - Max Iterations: 5 (лимит шагов)
 - Timeout: 30s (лимит времени)
- OpenAI Node → Parameters:
 - Max Tokens: 500 (лимит на ответ)
 - Model: gpt-4o-mini / gpt-4o (выбор в дропдауне)
- Три изменения → агент перестаёт сжигать деньги на простых запросах



AI Agent Node

Model	<code>gpt-4o-mini</code>
Max Iterations	5
Timeout (sec)	30
Max Tokens	500

[Demo](#)

ИТОГ блока 1:

Мышление — это ресурс. Им нужно управлять.

- У разных задач — разные требования к глубине рассуждения
- Параметр effort / reasoning управляет ценой и качеством
- Контекст растёт квадратично → нужны лимиты шагов
- 50–70% задач не нужна дорогая модель

БЛОК 2 — БЮДЖЕТЫ И ЗАЩИТА

Runaway agent который не может остановиться

Сценарии возникновения:

- Инструмент возвращает ошибку → агент уходит в retry бесконечно
- Цель не достижима → агент пробует разные стратегии, не останавливаясь
- Контекст переполнен → агент «не помнит», что уже делал этот шаг
- Баг в tool → агент заиклился на одном и том же действии

Кейс «пятничный деплой»:

14 000 API-вызовов, 380 млн токенов, счёт **\$12 400 за выходные**
CPU и RAM при этом в норме — только токены растут. Стандартный мониторинг не поможет.

Token DoS

может быть случайным или намеренным

Случайный Token DoS

Баг в коде → retry loop → тысячи запросов

Пользователь загружает огромный файл

RAG возвращает 100 документов вместо 5

API format change → агент не может распарсить

Намеренный Token DoS

Пользователь отправляет:
«Напиши эссе на 50 000
слов о каждом дне истории»

Цель — исчерпать ваш API-бюджет

Защита от обоих: одинаковая — бюджетные лимиты

Не нужно искать злоумышленника — случайный баг даёт тот же финансовый эффект

Бюджет агента

это четыре независимых лимита

Лимит шагов

`max_iterations = 10`

Останавливает
зацикленность

N шагов → стоп,
лог, уведомление

Лимит токенов

`max_tokens = 50 000`
на задачу

Прямое
ограничение
стоимости

Лимит времени

`timeout = 60 секунд`

Защита от зависших
tool calls

Лимит денег

`max_cost = $0.50` на запрос

Circuit breaker на уровне
бюджета

Ни один из четырёх не заменяет остальные — нужны все одновременно

Token Budget

15 строк кода, спасающих тысячи долларов

- Создаём budget tracker перед запуском агентной петли
- На каждом шаге трекаем input + output токены
- При превышении лимита → автоматически останавливаем
- Три уровня бюджетных лимитов:
 - На задачу: 50 000 токенов (~\$0.50)
 - На сессию: 200 000 токенов (~\$2.00)
 - На день / юзера: 1 000 000 токенов (~\$10)

```
class BudgetTracker:
    def __init__(self, max_tokens):
        self.max = max_tokens
        self.used = 0

    def track(self, inp, out):
        self.used += inp + out
        if self.used > self.max:
            raise BudgetExceeded(
                f"Лимит {self.max} токенов"
            )

# Три уровня лимитов:
TASK      = 50_000    # ~$0.50
SESSION   = 200_000   # ~$2.00
DAILY     = 1_000_000 # ~$10
```

Реальная 5-слойная защита в production

(9 агентов, 62 cron-задачи)

1

Timeout

Каждый cron-job → максимум 5 минут на задачу

2

Anti-loop

Max 3 retry, пауза 2 часа после превышения

3

Cost Circuit Breaker

\$15–20/день норма · \$50 Warning · \$100 Halt (автоостановка всех агентов)

4

Model Pinning

Дешёвые задачи не уходят на GPT-4o автоматически

5

Session Audit Log

Ручной разбор аномалий раз в неделю

Каждый слой ловит то, что пропускает предыдущий

1 Timeout → Разовый runaway session — агент завис на одном вызове

2 Anti-loop → Retry storm внутри пайплайна — повторные циклы

3 Cost CB → Медленный drift — рост расходов за несколько дней

4 Model Pinning → Конфигурационные баги при деплое — не та модель

5 Session Audit → Всё, что автоматика пропустила — аномалии в паттернах

Ни один слой не работает в одиночку — только система из пяти обеспечивает надёжность

Три метрики, которые ловят аномалии раньше алёрта

tokens_per_task

Норма:

исторический baseline

Аномалия:

рост в 2x
от baseline

Сигнал к немедленной
проверке агента

cost_per_completion

Норма:

\$0.05 / задача

Аномалия:

превышение baseline на 50%+

Автостоп при достижении порога

loop_iterations

Норма:

3–5 шагов

Аномалия:

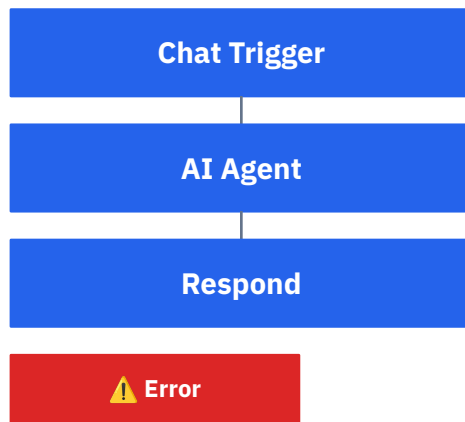
> 2x среднего
числа шагов

Silent bug detector

CPU и RAM при Token DoS в норме — нужны AI-специфичные метрики, стандартный Prometheus не поможет

На примере n8n бюджет настраивается без кода

- AI Agent Node → Max Iterations: 5
- Error Trigger Node при таймауте или ошибке — три действия:
 - Уведомление в Telegram / Slack с деталями ошибки
 - Лог в Google Sheets: timestamp, ошибка, входящий запрос
 - Stop workflow — предотвращает дальнейшие расходы
- Внешний мониторинг: n8n webhook → проверяет стоимость через OpenAI Usage API
- Error Workflow подключается через Settings → Error Workflow в настройках n8n



→ Telegram → Sheets → Stop

[Demo](#)

Агент должен уметь остановиться с частичным результатом

Early stopping — лучше чем hard error

Плохо: агент упирается в лимит → выбрасывает ошибку
→ пользователь видит «Error: budget exceeded»

Хорошо: агент видит остаток бюджета → при 20% остатке говорит:

«Завершаю с текущими данными» → возвращает частичный результат + статус

20% — порог для graceful degradation

Это UX-решение, а не только техническое

```
# Проверяем бюджет на каждом шаге
remaining = budget.max - budget.used

if remaining < 0.2 * budget.max:
    # Graceful degradation
    return {
        "result": agent.summarize_progress(),
        "status": "partial",
        "budget_used": budget.used,
        "message": "Завершено с частичным
результатом"
    }
```

Пользователь получает ценность даже при достижении лимита — а не просто ошибку

\$50 за 40 минут — и никто не заметил бы без мониторинга

Кейс — API format change

Что произошло:

- Изменили формат ответа одного инструмента
- Агент не смог распарсить ответ → запустил retry loop
- 200x рост токенов за 40 минут, \$50 списано незаметно
- Как обнаружили: только per-task token tracking
- Что спасает: алёрт «tokens_per_task > 2x baseline» → автоматический стоп

200x рост токенов за 40 минут · \$50 списано · CPU, RAM, error rate — в норме

ИТОГ блока 2:

Бюджет — это не экономия. Это стабильность.

- Runaway может случиться из-за бага, а не атаки
- Нужно 4 типа лимитов: шаги / токены / время / деньги
- 5-слойная защита ловит то, что пропускает 4-слойная
- Метрики должны быть AI-специфичными — CPU и RAM не показывают проблему
- Early stopping лучше hard error — пользователь получает ценность

БЛОК 3 — Авто-маршрутизация

Дорогая модель для всего = расточительство

- Типичная ошибка: один агент → одна модель (самая мощная)
- «Который час?» → GPT-5 → \$0.08
- «Напиши алгоритм» → GPT-5 → \$0.08

Без рутинга

Все запросы → GPT-5
Стоимость: \$0.08 / запрос

Разница в 16x на простом вопросе

С рутингом

Простые → GPT-5-mini
Стоимость: \$0.005 / запрос

16x экономия на том же вопросе

Разница в цене между Tier 1 и Tier 3 моделями: 17–19x

Роутер решает: какая модель нужна для запроса?



Цель: направить 80% запросов на дешёвую модель при сохранении 95%+ качества

Роутер может работать тремя способами

Rule-based

Ключевые слова / длина токенов /
тип задачи → cheap / expensive

- ✓ Просто
- ✓ Предсказуемо
- ✗ Негибко

Classifier-based

Отдельная мини-модель
оценивает сложность:
simple / complex / ambiguous

- ✓ Гибко
- ⚠ Нужны данные

Cascade

Сначала cheap → оцениваем
confidence → если < порога →
escalate к expensive

- ✓ Самая популярная
стратегия в production

Способ Cascade

сначала пробуем дёшево, при неуверенности — эскалируем

Запрос → Cheap Model (gpt-4o-mini)



Cheap Model → ответ + confidence score (0.0–1.0)

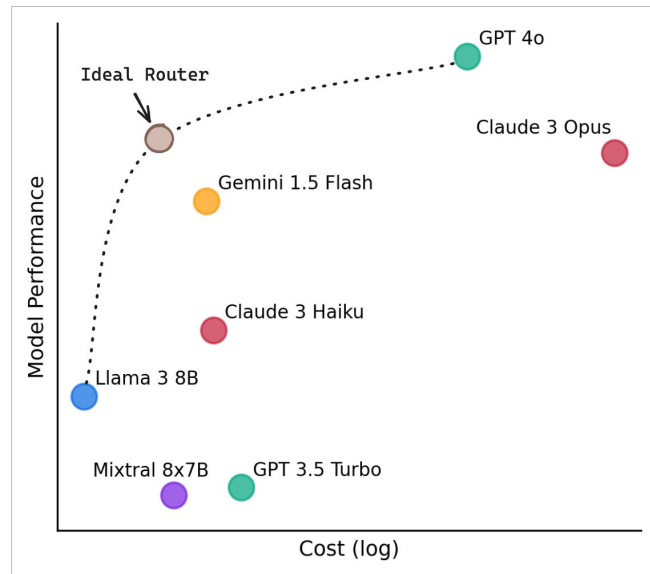


confidence ≥ 0.85

→ отдаём ответ ✓

confidence < 0.85

→ escalate → Expensive Model (gpt-4o) ✓



RouteLLM: 85% запросов на cheap · 95% качества от frontier · экономия 45–85%

Ценообразование с OpenRouter

Модель	Input (\$/M)	Output (\$/M)	Разница к Nano (input)	Лучше всего для
GPT-5.4 Nano	\$0.20	\$1.25	baseline	Классификация, извлечение, routing, sub-agents
GPT-5.4 Mini	\$0.75	\$4.50	3.75x Nano	Чат, coding assistants, агентные workflow
GPT-5.4	\$2.50	\$15.00	12.5x Nano	Сложный reasoning, long-context, multimodal
GPT-5.4 Pro	\$30.00	\$180.00	150x Nano	Максимальный reasoning, high-stakes задачи

Ценообразование с OpenRouter

Модель	Input (\$/M)	Output (\$/M)	Разница к Nano (input)	Tier
Claude Haiku 4.5	\$1.00	\$5.00	baseline	Tier 1
Claude Sonnet 4.6	\$3.00	\$15.00	3x Haiku	Tier 2
Claude Opus 4.6	\$5.00	\$25.00	5x Haiku	Tier 3
Opus 4.6 Long Ctx	\$10.00	\$37.50	10x Haiku	Tier 4

На классификации, форматировании, Q&A: gap между Tier 1 и Tier 3 < 2%

Триггеры роутинга

Триггеры эскалации на EXPENSIVE

- Confidence score < 0.85
- Агент запросил tool более 3 раз
- Ответ содержит «I'm not sure»
- Задача: complex / ambiguous
- Критический домен: медицина / юрист / финансы

Сигналы для CHEAP-модели

- Задача классифицирована как simple
- История: этот тип запросов cheap model решает хорошо
- Короткий запрос без специальных знаний
- Нет tool calls

Порог 0.85 — стартовое значение, нужно калибровать на своих данных

LLM оценивает уверенность через structured output

Как получить confidence score

Вариант 1 — в конце ответа: «CONFIDENCE: 0.72»

Вариант 2 — JSON mode: {"answer": "...", "confidence": 0.72, "reason": "requires domain expertise"}

Парсинг: RegExp или JSON.parse()

JSON mode надёжнее

```
You are a helpful assistant and a routing classifier.  
Answer the user's question briefly.  
Then on a new line write: CONFIDENCE: [score]
```

```
The score means: "How likely is it that a basic model is ENOUGH for  
this question?"
```

- 0.95-1.00 → Trivial. Single fact, translation, simple math, yes/no.
- 0.85-0.94 → Easy. Common knowledge, basic definitions, short explanation.
- 0.60-0.84 → Medium. Requires some reasoning or domain knowledge.
- 0.00-0.59 → Hard. Multi-step reasoning, expert domain, architecture design, comparing trade-offs, anything requiring depth or nuance.

```
IMPORTANT: Most questions that ask you to EXPLAIN, COMPARE, DESIGN,  
ANALYZE, or give EXAMPLES across multiple cases should score 0.59 or  
below.
```

```
Example scores:
```

- "Translate apple to Russian" → CONFIDENCE: 0.97
- "What is 2+2?" → CONFIDENCE: 0.99
- "What is the CAP theorem?" → CONFIDENCE: 0.72
- "Explain CAP theorem with 3 database examples" → CONFIDENCE: 0.45
- "Design microservice architecture for e-commerce" →
CONFIDENCE: 0.30

85% на дешёвой модели — без потери качества

Кейс — RouteLLM в production

85%

запросов остаются
на дешёвой модели

95%

качества от
baseline (потеря 2%)

45–85%

экономия от
исходных затрат

85% Haiku / mini — дешёвая модель

15% Opus

- До: 100% → Claude Opus → \$15/М токенов
- После: 85% → Claude Haiku (\$0.80/М) + 15% → Claude Opus (\$15/М)
- Потеря качества: ~2% — незаметна конечным пользователям

Роутер и бюджет работают вместе, а не по отдельности

Budget-aware routing

- При остатке $< 30\%$: принудительно роутим на cheap model + уведомляем пользователя о режиме экономии
- При остатке $< 10\%$: только простые запросы, сложные $\rightarrow$ в очередь или отклонить
- Система сама деградирует gracefully — а не ломается с ошибкой

```
if budget.remaining < 0.3 * budget.max:  
    force_route = "cheap"  
    notify_user("Режим экономии")
```

```
def route_with_budget(query, budget):  
    remaining = budget.remaining / budget.max  
  
    # Критический уровень  
    if remaining < 0.1:  
        if is_complex(query):  
            return queue_for_later(query)  
        return route_to("cheap", query)  
  
    # Экономный режим  
    if remaining < 0.3:  
        notify("Режим экономии активен")  
        return route_to("cheap", query)  
  
    # Нормальный режим — роутер решает  
    confidence = get_confidence(query)  
    if confidence >= THRESHOLD:  
        return route_to("cheap", query)  
    return route_to("expensive", query)
```

Правильный порог — это эксперимент, а не угадывание

Калибровка — главная сложность роутинга

Порог 0.95 (высокий)

Большинство уходит
на дорогую модель

Экономии нет

Ошибка: перекалибровали
в сторону качества

Порог 0.85 (оптимум ★)

85% на cheap
15% на expensive

Качество 95%+

Старт с этого значения,
затем настроить

Порог 0.60 (низкий)

Большинство на cheap

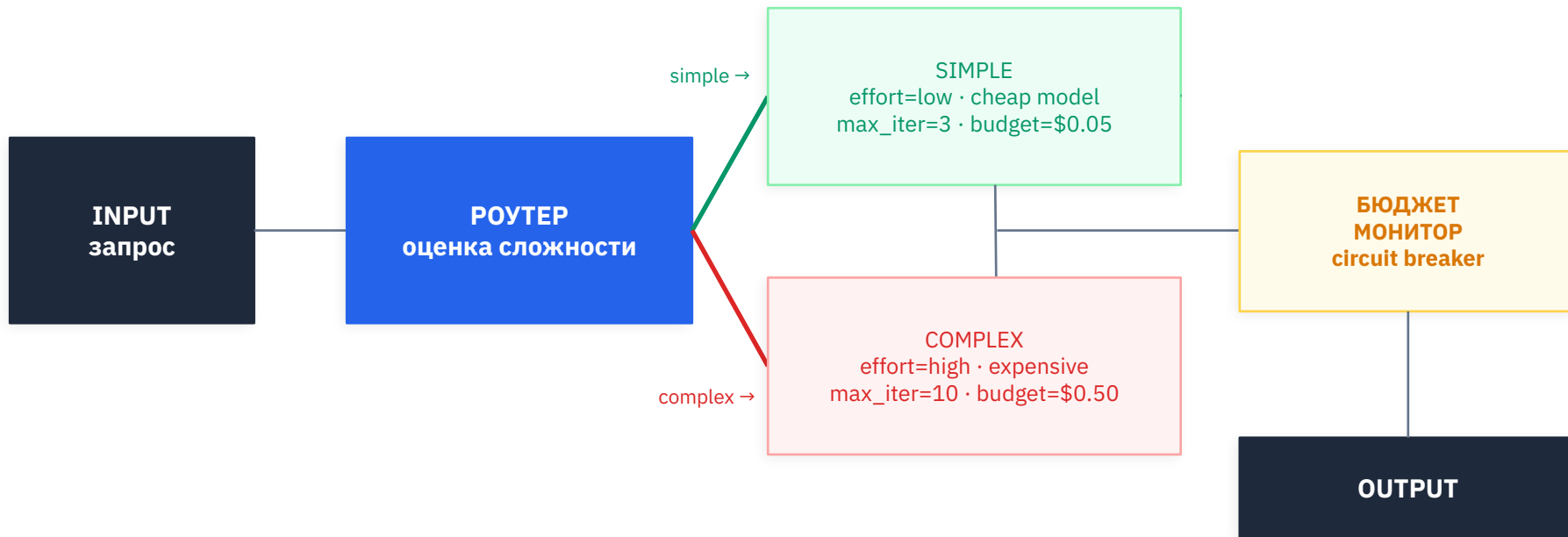
Качество падает
Пользователи недовольны

Ошибка: недокалибровали

Запустить оба варианта на 100–200 реальных запросах · LLM-Judge оценивает · найти max(экономия) при quality ≥ цели

Все три механизма вместе — главная архитектура

Полная схема «умного» агента



ИТОГ блока 3:

Авто-маршрутизация меняет unit economics агента

- 80% запросов дешёвая модель решает с качеством 95%+
- Cascade: сначала cheap → escalate по confidence триггерам
- Confidence score — главный сигнал для эскалации
- Роутер + бюджет = система, деградирующая gracefully
- Порог нужно калибровать через эксперимент — не на глаз

Практика

Узел 1: AI Agent (gpt-4o-mini) + системный промпт:
«Ответь и укажи уверенность 0–1»

Узел 2: Code Node → извлечь число из
«CONFIDENCE: 0.72»

Узел 3: IF Node → confidence $\geq$ 0.85?

- ДА → Output → пользователь
- НЕТ → AI Agent (gpt-4o) → Output

```
// Code Node — парсим confidence
const text = $input.first().json.output;

const match = text.match(
  /CONFIDENCE:\s*([\d.]+)/
);

const confidence = match
  ? parseFloat(match[1])
  : 0.5;

return { confidence };
```

IF Node

confidence $\geq$ 0.85

TRUE → Output

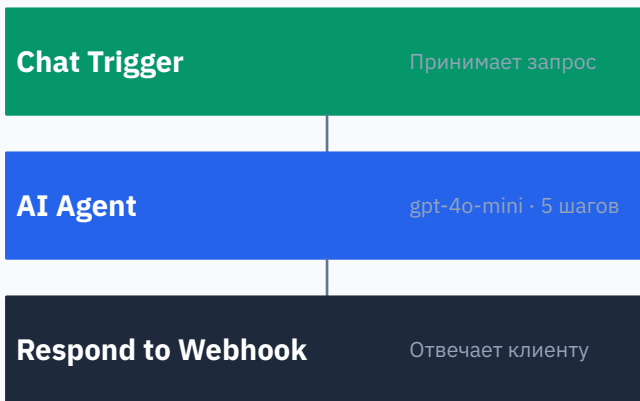
FALSE → GPT-4o

[Demo](#)

Шаг 1 — создаём скелет workflow в n8n

1. Добавить узлы: Chat Trigger → AI Agent → Respond to Webhook
2. AI Agent: модель gpt-4o-mini, Max Iterations: 5

Workflow в n8n



Шаг 2 — system prompt

- Добавить в ноду AI Agent System prompt
-  Проверить: простой вопрос → confidence высокий, сложный → низкий

0.85 — стартовый порог

Настроить после теста на реальных запросах

```
You are a helpful assistant and a routing classifier.  
Answer the user's question briefly.  
Then on a new line write: CONFIDENCE: [score]
```

```
The score means: "How likely is it that a basic model is ENOUGH for  
this question?"
```

```
- 0.95-1.00 → Trivial. Single fact, translation, simple math,  
yes/no.  
- 0.85-0.94 → Easy. Common knowledge, basic definitions, short  
explanation.  
- 0.60-0.84 → Medium. Requires some reasoning or domain knowledge.  
- 0.00-0.59 → Hard. Multi-step reasoning, expert domain,  
architecture design, comparing trade-offs, anything requiring depth  
or nuance.
```

```
IMPORTANT: Most questions that ask you to EXPLAIN, COMPARE, DESIGN,  
ANALYZE, or give EXAMPLES across multiple cases should score 0.59 or  
below.
```

```
Example scores:
```

```
- "Translate apple to Russian" → CONFIDENCE: 0.97  
- "What is 2+2?" → CONFIDENCE: 0.99  
- "What is the CAP theorem?" → CONFIDENCE: 0.72  
- "Explain CAP theorem with 3 database examples" → CONFIDENCE: 0.45  
- "Design microservice architecture for e-commerce" →  
CONFIDENCE: 0.30
```

Шаг 3 — парсим confidence и проверяем IF-ом

- Добавить Code Node после AI Agent
- IF Node: условие confidence ≥ 0.85
 - ДА → Output (ответ достаточно хорош)
 - НЕТ → следующий узел (эскалация)
-  Проверить: простой вопрос → confidence высокий, сложный → низкий

```
// IF Node — условие
{{ $json.confidence >= 0.85 }}
```

```
// Code Node
const text = $input.first().json.output
  || $input.first().json.text
  || "";

const match = text.match(
  /CONFIDENCE:\s*([0-9]+\.[0-9]*)/
);

const confidence = match
  ? parseFloat(match[1])
  : 0.5; // fallback

return {
  json: {
    output: text,
    confidence: confidence,
  }
};
```

Шаг 4 — эскалация на GPT-4o при low confidence

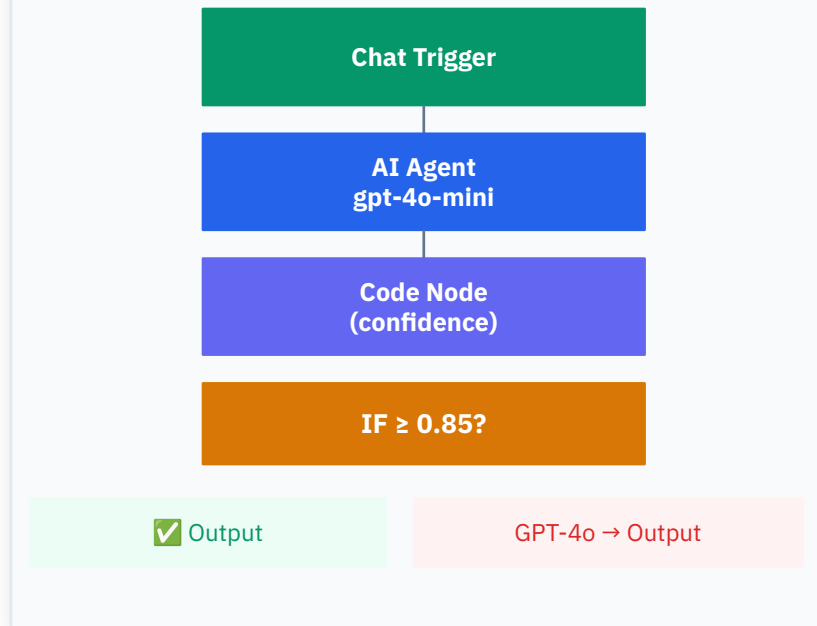
В ветку НЕТ добавить: AI Agent (gpt-4o), Max Iterations: 10

Проверить на двух запросах:

«Как дела?» → cheap model, confidence ~0.95

«Объясни CAP-теорему с примерами» → escalate

Финальный workflow



Результат

Запрос	Новый ожидаемый confidence	Результат
«Как переводится apple?»	0.95–0.99	cheap ✓
«Столица Франции?»	0.97–0.99	cheap ✓
«Что такое теорема CAP?»	~0.72	cheap ✓
«Объясни CAP с 3 примерами БД»	~0.45	escalate ✓
«Теория струн» (простое описание)	~0.68	cheap ✓
«Архитектура микросервисов»	~0.30	escalate ✓

Шаг 5 — тестируем и замеряем экономию

Отправить 10 тестовых запросов:

Простые (cheap): «Привет», «Переведи слово» и тп

Сложные (escalate): «Напиши план проекта»,
«Объясни алгоритм Дейкстры»

Замерить: сколько % ушло на cheap model?

Ожидаемый результат: $\geq 60\%$ запросов на cheap модели

Запрос	Модель	conf.
Привет	mini ✓	0.97
Переведи слово	mini ✓	0.95
Сколько 2+2	mini ✓	0.99
Классификация	mini ✓	0.88
Объясни CAP	gpt-4o	0.61
Алгоритм Дейкстры	gpt-4o	0.58
План проекта	gpt-4o	0.52

Запомните три вещи

Мышление — ресурс

effort / reasoning / model tier

Управляйте явно,
а не по умолчанию

Чем дешевле усилие —
тем важнее точность
настройки

Бюджет — страховка

4 типа лимитов
+ circuit breaker (стоп-кран)

Агент не сожжёт
деньги, даже если
что-то пойдёт не так

Early stop > hard error*

Роутинг меняет экономику

80% запросов → cheap model

95%+ качества

45–85% экономии без потери UX

это не оптимизация, а
архитектура

***Early stop** — агент сам замечает, что приближается к лимиту (токены, шаги, бюджет), и завершает работу корректно: возвращает частичный результат, объясняет что не успел, передаёт управление дальше.

Hard error — агент не останавливается заранее, врывается в жёсткий лимит API или исчерпывает бюджет, после чего падает с исключением, ломая весь пайплайн.

Топ-5 ошибок, которые стоят денег



Одна модель на все задачи без роутинга

Платите 17–19x за задачи, которые решает дешёвая модель



Нет лимита итераций в агентной петле

\$12 400 за выходные — реальный кейс без `max_iterations`



Отслеживать CPU/RAM, но не токены

Token DoS незаметен в стандартных дашбордах Prometheus



Hard error вместо graceful degradation

Пользователь теряет ценность вместо частичного результата



Порог confidence взять «на глаз», не калибровать

Слишком высокий или слишком низкий — оба варианта плохи



Спасибо за внимание

Вопросы? — 10–15 минут